Journal of The Korea Society of Computer and Information Vol. XX No. X, pp. 0-0, Month 20XX
https://doi.org/10.9708/jksci.20XX.XX.XX.000

GPS-PromptOps: Genetic Prompt Search for JOILang Code Generation on Local LLMs

Mingi Jeong[1,2], Eunjin Jeong[2], Soonhoi Ha[2]

[1]Electronics and Telecommunications Research Institute (ETRI), Daejeon, Korea
[2]Department of Computer Science and Engineering, Seoul National University, Seoul, Korea

- First Author: Mingi Jeong, Corresponding Author: Soonhoi Ha
- Mingi Jeong (mgjeong@etri.re.kr), Electronics and Telecommunications Research Institute (ETRI)
- Eunjin Jeong (chjej202@snu.ac.kr), Department of Computer Science and Engineering, Seoul National University
- Soonhoi Ha (sha@snu.ac.kr), Department of Computer Science and Engineering, Seoul National University
- Received: 20XX. XX. XX, Revised: 20XX. XX. XX, Accepted: 20XX. XX. XX

[Abstract] JOILang is an IoT domain-specific language (DSL) designed to describe user-defined automation scenarios at the service level. This paper proposes a technique to optimize a JOILang prompt, originally written for cloud-based LLMs, for individual local LLMs by automatically transforming it through a Genetic Algorithm (GA), thereby converting natural language commands into JOILang code. GPS-PromptOps represents a prompt as a combination of role-specific blocks and their order, detailed rules, examples, and generation settings. For each candidate, the target model generates JOILang code, which a rule-based evaluator checks for registered device functions and their arguments, JSON format, and temporal conditions. GPS-PromptOps then selects and recombines blocks from higher-scoring candidates and revises related blocks according to the reported failure types. We compared GPS-PromptOps with DirectTransfer, which applies the original prompt unchanged, on three local models: Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, and Qwen2.5-Coder-14B-Instruct. DETPass denotes the proportion of benchmark items that pass the rule-based evaluation. On the benchmark set used for search and evaluation, GPS-PromptOps improved DETPass by 3.9-7.0 percentage points over DirectTransfer, with a mean improvement of 5.2 points. Mean per-item token count and latency decreased by 65.1% and 43.7%, respectively. When the selected prompts were exchanged across the three models, each model achieved its highest DETPass with its own prompt, exceeding the best prompt selected for another model by 2.7 points on average. A rescoring-based sensitivity analysis further shows that the reported comparison is robust to ±20% perturbations of the evaluation weights. These results demonstrate the necessity of optimizing prompts according to the specific local LLM model and highlight the effectiveness of the proposed technique.

▶ Key words: genetic prompt search, PromptOps, DET evaluation, local LLMs, JOILang, model-specific prompt adaptation

[요 약] JOILang은 서비스 수준에서 사용자가 원하는 자동화 시나리오를 기술하기 위한 IoT 도메인 특화 언어(domain-specific language, DSL)이다. 본 논문은 자연어 명령을 JOILang 코드로 바꾸기 위해, 클라우드 LLM용으로 작성한 JOILang 프롬프트를 진화 알고리즘(GA)을 이용하여 자동 변환함으로써 로컬 LLM별로 최적화하는 기법을 제안한다. 제안하는 GPS-PromptOps는 프롬프트를 역할별 블록과 배치 순서, 세부 규칙, 예제 및 생성 설정의 조합으로 표현한다. 각 프롬프트 후보를 대상 모델에 적용하여 JOILang 코드를 생성하고, 등록된 장치 기능과 인자, JSON 형식 및 시간 조건을 확인하는 규칙 기반 평가기로 검사한다. GPS-PromptOps는 점수가 높은 후보의 블록을 선택·재조합하고, 평가기가 반환한 실패 유형에 따라 관련 블록을 수정한다. Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, Qwen2.5-Coder-14B-Instruct의 세 로컬 모델에서 클라우드 LLM을 기준으로 작성한 프롬프트를 그대로 적용한 DirectTransfer와 GPS-PromptOps를 비교하였다. DETPass는 규칙 기반 평가를 통과한 평가 항목의 비율이다. 탐색과 평가에 사용한 벤치마크 집합에서 GPS-PromptOps의 DETPass는 DirectTransfer보다 3.9-7.0 %p 높았으며, 평균 차이는 5.2 %p였다. 평가 항목당 평균 토큰 수와 평균 지연 시간은 각각 65.1%와 43.7% 감소하였다. 선택된 프롬프트를 세 모델에 서로 바꾸어 적용했을 때에도 각 모델은 자체 탐색 프롬프트에서 가장 높은 DETPass를 보였으며, 최선의 타 모델용 프롬프트보다 평균 2.7 %p 높았다. 저장된 생성 결과를 재채점하는 민감도 분석에서는 평가 가중치의 ±20% 변형에 대해 비교 결과가 강건함을 확인하였다. 이 결과는 로컬 LLM 모델에 따라 프롬프트를 최적화하는 것이 필요함과 제안하는 기법이 유용함을 보여준다.

▶ 주제어: JOILang, 유전적 프롬프트 탐색, PromptOps, DET 평가, 로컬 LLM, 모델별 프롬프트 적응

I. 서론

서비스 기반 IoT 플랫폼은 사용자가 스마트 기기를 직접 제어하는 대신, 사용자가 원하는 서비스를 기술하면 서비스를 수행하는 스마트 기기를 찾아서 제어한다 [1]. JOILang은 서비스 수준에서 사용자가 원하는 자동화 시나리오를 기술하기 위한 IoT 도메인 특화 언어(domain-specific language, DSL)이다 [2]. 서비스는 온도·습도·문 열림 여부처럼 장치의 상태나 센서값을 읽는 VALUE

표 1. 자연어–JOILang 변환의 주요 제약과 검증 항목

제약 유형	검증 항목
서비스 스키마 일치	서비스의 정식 명칭, 인자 자료형과 허용값이 스키마와 일치하는가
서비스 유형 구분	상태·센서값 읽기와 장치 제어를 올바르게 구분했는가
대상 장치 및 태그 범위	공간·소유·대상 태그와 하나 또는 전체 장치를 가리키는 `any/all` 범위가 의도와 일치하는가
상태 조건과 상태 전이	현재 상태를 검사하는 `if`와 사건을 기다리는 `wait until`을 구분했는가
상대 시간의 기준	후속 지연의 기준 시점이 자연어 명령의 순서와 일치하는가
지연 구문과 단위	승인된 시간 구문과 밀리초 단위를 사용했는가
실행 주기와 반복	단일 실행, 고정 일정, 반복 감시를 올바르게 구분했는가
출력 형식	필수 키를 갖춘 하나의 JSON 객체인가

표 2. 예비 실험에서 관찰한 모델별 프롬프트 민감도

모델	영향이 컸던 블록	주요 관찰 결과
Qwen2.5-Coder-7B	장치·기능 정보, 시간 조건 규칙	서비스의 정식 명칭을 앞쪽에 제시하면 등록되지 않은 서비스를 사용하는 오류가 줄었고, 시간 조건 규칙을 강화하면 시간 오류가 감소하였다.
Llama-3.1-8B	출력 형식 규칙, 예제 순서	JSON 출력 형식 규칙을 앞쪽에 배치했을 때 형식 오류가 줄었으며, 예제 순서에 따라 결과 변동이 크게 나타났다.
Qwen2.5-Coder-14B	예제 수, 프롬프트 길이	예제를 늘리면 일부 명령에서 정확도가 개선되었지만, 긴 프롬프트에서는 토큰 수와 지연 시간이 증가하였다.

서비스와, 조명을 켜거나 에어컨의 설정 온도를 바꾸는 등의 제어 동작을 기술하는 FUNCTION 서비스로 구분된다. 서비스를 수행할 스마트 기기의 범위는 기기를 플랫폼에 등록할 때 명시하는 태그로 한정할 수 있다. 태그는 사용자가 지정한 공간이나 기기의 유형을 지정하는데 사용할 수 있는데, 예를 들어 `(#living #AirConditioner)` 태그는 거실에서 에어컨 기능을 제공하는 장치를 의미한다. 이와 같은 서비스 단위 표현은 기기의 제조사와 동작 방법이 바뀌어도 사용자가 기술한 자동화 시나리오는 변경할 필요가 없다는 장점을 갖는다 [3].

LLM 기술의 발전으로 사용자가 원하는 자동화 시나리오를 자연어로 표현하면 JOILang 코드를 자동생성하는 것이 가능하게 되었다. 코드 자동 생성은 일반적인 언어 변환과 달리 엄격한 요구 사항을 만족시켜야 한다. 생성 코드가 정의된 서비스 이름을 정확히 사용했는지, 인자의 자료형과 허용값이 맞는지, 대상 태그와 시간 조건을 올바르게 표현했는지 등 문법이 점검되어야 한다. 잘못된 서비스를 요구하거나, 잘못된 공간의 장치를 지정하면 실제 자동화에서 오동작이 발생할 수 있다. 표 1은 자연어–JOILang 변환 과정에서 다루는 제약과 검증 항목을 정리한 것이다.

자연어–JOILang 변환에서 또 하나의 어려운 점은 자연어의 표현의 모호성을 고려해야 한다는 것이다. 예를 들어 “문이 닫혀 있으면 사이렌을 켜 줘”는 현재 상태를 확인하는 조건이고, “문이 닫히면 사이렌을 켜 줘”는 문이 열린 뒤 다시 닫히는 사건을 기다리는 조건이다. 전자는 `if`로 현재 상태를 확인하고, 후자는 `wait until`으로 상태 전이를 기다린다. 두 조건은 실행 시점과 실행 횟수가 다르므로 자연어의 표현을 정확히 구분해야 한다. 이러한 모호성을 극복하기 위해, 상당한 양의 명령–코드 예제를 프롬프트에 포함시킬 필요가 있다. 상대 시간 표현에서도 지연의 기준 시점이 앞선 사건인지 직전 동작인지에 따라 실행 결과가 달라질 수 있다.

선행 연구에서는 자연어 명령을 JOILang 코드로 자동 변환하는 파이프라인을 제시하였다 [2]. 이 파이프라인은 먼저 사용자가 요구하는 서비스를 기반으로 장치·기능 정보를 찾는다. 서비스 스키마는 사용할 수 있는 장치 기능의 정식 명칭과 인자의 자료형·허용값을 정리한 목록이다. 파이프라인은 전체 스키마에서 해당 명령과 관련된 항목을 추려 프롬프트에 제공한다. LLM이 이 정보와 JOILang 문법 및 명령 – 코드 예제를 바탕으로 JSON 형식의 시나리오를 생성하면, 규칙 검사와 수정 절차로 결과를 보완한다.

그림 1은 자연어 명령과 관련 장치·기능 정보가 JOILang 시나리오로 변환되는 예를 보여 준다. 그림에서 `#living`과 `#AirConditioner`는 사용자 지정 공간과 서비스 유형을 함께 나타내며, 실행 시점에 실제 장치와 연결된다. 연속된 두 `wait until`은 문이 열린 뒤 다시 닫히는 상태 전이를 표현하고, `clock_delay`는 밀리초 단위의 지연을 나타낸다. 결과는 `name`, `cron`, `period`, `code` 필드를 갖는 하나의 JSON 객체로 출력된다. 여기서 `cron`과 `period`는 실행 일정과 반복 조건을, `code`는 실행할 명령을 담는다.

자연어–JOILang 변환를 위한 프롬프트의 크기는 시스템에 등록된 스마트 기기의 종류와 개수 뿐 아니라 명령–코드 예제의 양에 따라 증가한다. 기존의 연구는 클라우드 기반 LLM을 사용하는데, 프롬프트가 길어지면 토큰 비용이 증가하고, 인터넷 조건에 따라 응답 지연 시간이 늘어날 수 있다는 단점이 있다. 아울러 사용자 명령과 장치 목록 및 센서 상태가 외부로 전송된다는 우려가 있다. 이러한 문제를 해결하기 위해 본 논문에서는 엣지 환경에서 로컬 LLM을 이용하여 자연어를 JOILang 코드로 변환하는 문제를 다룬다.

엣지 환경에서는 사용할 수 있는 연산량과 메모리가 제한되고, 하드웨어 플랫폼마다 성능도 다르기 때문에 클라우드 LLM 기반 변환 방법을 그대로 사용할 수 없다 [4, 5]. 아울러 로컬 LLM마다 문맥 길이, 토큰 처리 방식, 코드 생성 능력, 지시 이행과 예제 활용 방식이 다르므로, 같은 프롬프트를 사용하는 경우에도 등록되지 않은 서비스 사용, 시간 조건 누락이나 출력 형식 오류가 발생할 수 있다. 선행연구도 소수의 명령 – 코드 예제, 프롬프트 형식, 예제 선택과 순서가 언어 모델 출력에 큰 영향을 줄 수 있음과 [6]. 모델마다 효과적인 예제 순서와 형식이 달라질 수 있음을 보였다 [7, 8].

선행 연구 결과를 확인하기 위해 크기를 줄인 동일한 클라우드 기반 프롬프트를 여러 로컬 LLM에 적용하는 예비 실험을 수행하였다. 예비 실험에서는 프롬프트를 모델 역할, 장치·기능 정보, JOILang 문법, 시간 조건, 출력 형식과 명령 – 코드 예제의 역할별 구성 요소로 나누고, 각 구성 요소의 위치와 길이를 바꾸었다. 이러한 역할별 구성 요소를 프롬프트 블록이라 부른다. 표 2에서 요약한 바와 같이 같은 자연어 명령과 서비스 스키마를 제공해도 블록의 배치와 길이에 따라 모델별 오류 유형과 성능 변화가 달라진 것을 관찰할 수 있었다.

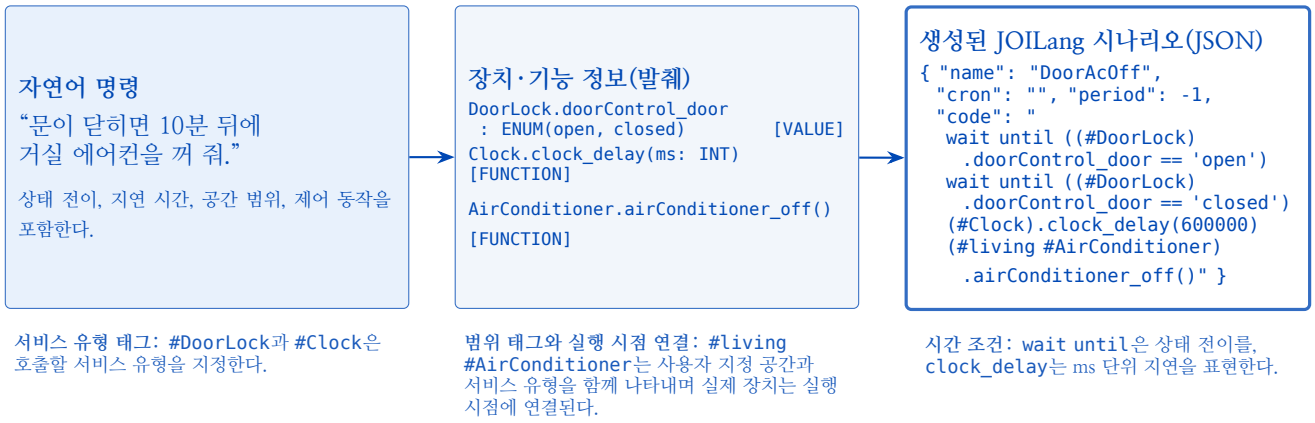


그림 1. 자연어 명령과 장치·기능 정보로부터 JOILang 시나리오를 생성하는 예. 생성 코드는 제공된 서비스 스키마, 대상 범위, 시간 조건 및 JSON 출력 형식을 만족해야 한다.

로컬 LLM마다 효과적인 프롬프트 구성이 다를 수 있으므로, 클라우드에서 작성한 JOILang 프롬프트를 대상 모델의 특성에 맞게 조정할 필요가 있다. 그러나 모델별로 적합한 프롬프트 구성을 수작업으로 찾기에는 어려움이 있다. 본 논문은 이 조정 과정을 프롬프트 블록의 조합을 찾는 탐색 문제로 정의하고, 모델별 구성을 자동으로 탐색하는 방법을 제안한다. 클라우드 프롬프트는 모델 역할, 장치·기능 정보, JOILang 문법, 시간 조건, 출력 형식, 등록되지 않은 서비스의 사용을 제한하는 규칙, 명령–코드 예제, 수정 규칙 및 생성 설정을 담당하는 블록 등으로 구성한다. 각 블록은 사용 여부를 바꿀 수 있으며, 설명 길이와 예제 수도 조정할 수 있어 가능한 조합의 수가 빠르게 증가한다. 또한 오류를 줄이기 위해 설명이나 규칙을 보강하면 프롬프트가 길어지는 반면, 프롬프트를 줄이면 토큰 사용량은 감소하지만 코드 생성에 필요한 규칙이 누락될 수 있다. 이처럼 생성 정확도와 실행 비용을 함께 고려하면서 대상 모델에 적합한 블록 조합을 찾는 것이 본 연구에서 다루는 핵심 문제이다.

이러한 조합을 사람이 수작업으로 평가하기는 어렵다. 개별 후보의 성능은 로컬 LLM으로 JOILang 코드를 생성한 뒤 규칙 검사와 토큰 수·지연 시간 측정을 마쳐야 확인할 수 있기 때문이다. 본 연구는 후보의 성능을 자동으로 평가하고, 성능이 좋은 후보의 프롬프트 블록을 재조합하며, 실패한 항목과 관련된 블록을 바꾸어서 더 좋은 조합을 찾아가는 유전 알고리즘(genetic algorithm, GA)을 사용한다. 우리가 제안하는 GA 기반 탐색·선택 방법을 GPS–PromptOps라고 부르며 다음 2가지 기법과 비교한다. CloudRef는 클라우드 모델에서 원 프롬프트를 실행한 기준 구성이고, DirectTransfer는 같은 프롬프트를 로컬 모델에 그대로 적용한 기준선이다.

본 논문에서는 생성 코드가 서비스 스키마, 대상 태그, 시간 조건과 출력 형식을 따르는지 확인하는 절차를 DET 평가라고 한다. 주요 평가지표는 DETPass와 평가 항목당 평균 토큰 수, 그리고 지연 시간이다. DETPass는 DET 평가를 통과한 항목의 비율이다. GA는 규칙 평가 결과와 토큰 수·지연 시간을 합친 종합 점수인 적합도로 후보를 비교하고, 전체 탐색에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 선택한다.

본 논문의 주요 기여를 요약하면 다음과 같다.

- 엣지 LLM을 이용하여 자연어를 JOILang 코드로 변환하는 문제를 클라우드 기반 프롬프트를 로컬/엣지 LLM에 맞게 조정하는 블록 단위 프롬프트 탐색 문제로 정의한다.
- 사용할 블록, 세부 규칙, 예제 수와 생성 설정을 포함하는 프롬프트 표현 방식을 설계하고, 이를 최적화하는 GA 기반 PromptOps 절차를 제안한다.
- 규칙 평가 결과와 토큰 수 및 지연 시간을 후보 적합도에 반영하고, 실패 유형에 따라 관련 블록을 수정하여 모델별 최종 프롬프트를 선택한다.
- Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, Qwen2.5-Coder-14B-Instruct의 세 로컬 모델에서 GPS-PromptOps를 CloudRef 및 Direct-Transfer와 비교하고, 최종 프롬프트의 교차 적용과 평가 가중치·적합도 계수의 민감도 분석을 통해 DETPass, 토큰 효율과 모델별 차이를 평가한다.

본 논문의 구성은 다음과 같다. II장에서는 관련 연구와 배경을 정리하고, III장에서는 JOILang 프롬프트의 블록 구성과 탐색 공간을 정의한다. IV장에서는 GA 기반 PromptOps 절차를 설명하고, V장에서는 실험 결과를 보고한다. 마지막으로 VI장에서 결론과 향후 연구를 제시한다.

II. 관련 연구와 배경

1. 배경지식: JOILang

JOILang은 자연어로 표현된 IoT 자동화 시나리오를 서비스 단위 실행 코드로 변환하기 위한 DSL이다 [2]. JOILang 시나리오는 시나리오 이름, 실행 주기, 실행 코드 블록, 서비스 참조·호출과 시간 조건 표현을 포함한다. 자연어로 부터 자동 생성된 JOILang 코드는 등록된 서비스와 인자 규칙, 대상 태그, 시간 조건 및 JSON 출력 형식을 함께 만족해야 한다.

이러한 특성 때문에 JOILang 생성은 일반적인 지시문 생성 문제와 다르다. 생성 결과가 서비스 스키마와 실행 규칙을 만족해야 하므로 의미 유사도만으로 품질을 판단하기 어렵고 규칙 기반 평가가 필요하다. 또한 자연어 명령, 장치·기능 정보, 시간 조건과 출력 형식이 함께 작용하므로 프롬프트를 역할별 블록으로 나누어 관리할 필요가 있다.

2. 프롬프트 민감도와 로컬 모델 적응

기존 연구는 프롬프트의 형식, 예제 선택, 예제 순서가 언어 모델의 출력에 큰 영향을 줄 수 있음을 보였다 [6–8]. 한 모델에서 효과적인 프롬프트 구성이 다른 모델에서도

동일하게 효과적이라고 보기 어렵다는 것이다. 이에 클라우드에서 작성한 JOILang 프롬프트를 로컬 LLM에 그대로 적용하는 방식(DirectTransfer)이 효율적이 아님을 예상할 수 있다.

클라우드 기반 JOILang 프롬프트는 장치·기능 정보, 문법, 시간 조건, 출력 형식과 명령–코드 예제처럼 역할이 다른 블록으로 구성된다. 어떤 블록을 포함하고 얼마나 자세히 설명하는지에 따라 모델의 출력이 달라질 수 있다. 로컬 LLM마다 문맥 처리, 지시 이행, 코드 생성과 예제 활용 방식이 다르므로, 본 연구는 블록 구성을 대상 모델별로 조정한다.

3. 자동 프롬프트 최적화와 진화 기반 탐색

최근 프롬프트 최적화 연구는 사람이 직접 문구를 조정하는 방식에서 벗어나, 알고리즘으로 후보를 탐색하는 방향으로 확장되고 있다. AutoPrompt [9]는 입력 앞에 덧붙일 토큰 조합을 자동으로 찾았고, APE [10]는 여러 후보 지시문을 생성한 뒤 점수로 선택하는 방법을 제시하였다. GrIPS [11]는 모델의 출력 점수를 바탕으로 지시문을 편집하며, OPRO [12]는 이전 후보와 점수를 LLM에 제공하여 새 지시문을 반복적으로 생성한다. EvoPrompt와 Promptbreeder는 여러 후보를 함께 유지하고 조합·수정하는 진화 탐색을 프롬프트 최적화에 적용하였다 [13, 14].

기존의 연구는 주로 일반 자연어처리·추론 과제나 단일 지시문을 탐색하는 반면 본 연구는 문장 전체를 임의로 바꾸는 대신 프롬프트 블록을 조합하고, DET 평가의 실패 유형에 따라 관련 블록의 세부 규칙을 수정한다는 점에서 차이가 있다.

4. 피드백 기반 개선

Self-Refine과 Reflexion은 LLM이 자신의 출력이나 행동 결과에 대한 피드백을 이용해 다음 생성을 개선할 수 있음을 보였다 [15, 16]. Self-Refine은 같은 모델이 초기 출력에 대한 피드백을 만들고 이를 바탕으로 출력을 반복 수정한다. Reflexion은 환경에서 얻은 실패와 보상 결과를 글로 정리해 저장하고 다음 시도에 활용한다. 본 연구는 생성 결과를 다시 작성하는 대신, 규칙 기반 검증에서 확인한 실패 유형을 프롬프트 블록의 변이와 후보 선택에 사용한다.

5. 제약 기반 생성과 규칙 기반 평가

DSL 생성에서는 의미 유사도만으로 출력 품질을 판단하기 어렵다. 문법적으로 그럴듯한 출력도 서비스 스키마에 없는 기능을 사용하거나 인자 자료형과 허용값, 안전 정책 및 실행 제약을 어길 수 있기 때문이다. PICARD는 text-to-SQL 생성에서 점진적 구문 분석을 이용해 허용되지 않는 토큰을 디코딩 과정에서 제거하였다 [17]. 문법 제약 디코딩 연구는 형식 문법으로 구조화된 출력의 생성 범위를 제한할 수 있음을 보였다 [18]. 이 연구는 형식과 스키마를 만족해야 하는 구조화 출력을 다룬다는 점에서 JOILang 생성과 관련된다. 하지만 PICARD와 문법 제약 디코딩은 주로 생성 과정에서 잘못된 토큰을 제한하는 반면 본 연구는 로컬 LLM의 내부 생성 방식을 바꾸지 않고, 생성 후의 DET 평가 결과를 후보 점수와 블록 수정에 사용한다는 점에서 차이가 있다.

6. 프롬프트 압축, LLM 서빙, PromptOps

프롬프트 압축과 LLM 서빙 연구는 토큰 수, 지연 시간과 처리량을 줄이는 방법에 초점을 맞추고 있다. LLMLingua는 긴 프롬프트를 단계적으로 압축하여 추론 비용을 낮추었고, vLLM은 PagedAttention을 이용해 모델 추론 중 저장되는 중간 계산 결과의 메모리를 효율적으로 관리하였다 [19, 20]. 이러한 연구는 토큰 수와 지연 시간을 평가 기준에 포함하는 근거를 제공한다.

그러나 프롬프트를 줄이는 과정에서 서비스의 정식 명칭이나 인자 자료형 오류, 시간 조건 누락, JSON 형식 위반이 늘어나면 JOILang 생성 품질이 낮아진다. 본 연구는 DET 평가로 후보의 정확성을 비교하고, 선택한 프롬프트의 토큰 수와 지연 시간을 별도로 보고한다.

표 3은 관련 연구군을 JOILang 프롬프트의 로컬 적응에 필요한 요구사항별로 비교한다. 기존 연구는 자동 탐색, 피드백 활용, 구조화 출력 제약, 비용 측정 중 일부를 다루지만, 서비스 스키마를 따라야 하는 DSL에서 블록 구성, 모델별 적응, DET 평가 및 토큰·지연 시간을 함께 고려한 후보 선택을 하나로 다루지는 않는다.

III. JOILang 프롬프트 구성과 탐색 공간

1. 프롬프트 블록 구성과 확장

본 연구의 출발점은 클라우드 LLM에서 검증된 JOILang 프롬프트이다. 프롬프트는 단일한 긴 문장이 아니라 여러 프롬프트 블록의 조합으로 구성된다. DirectTransfer는 클라우드 프롬프트를 로컬 모델에 그대로 적용한 방법이고, 제안하는 GPS-PromptOps는 GA 알고리즘을 이용해서 각 로컬 대상 모델에 맞게 블록 구성을 조정하는 방법이다.

표 4는 클라우드 프롬프트가 어떤 프롬프트 블록으로 구성되며, 각 블록의 역할이 무엇이고, 후보 간에 무엇이 달라지는지를 정리한 것이다. 역할·과제 설명 블록은 고정이고, 서비스 스키마 블록에서 스키마 항목은 명령별 장치·기능 검색 결과에 따라 달라지지만 같은 평가 항목의 모든 후보에 동일하게 제공되므로 GA가 변경하지 않는다.

JOILang 문법은 `if`, `wait until`, `clock_delay`, 허용값, 단위 변환, 대상 태그, 소유자와 장치의 관계, JSON 전용 출력과 같은 세부 규칙으로 나눌 수 있다. 평가기는 `unknown_service`(서비스 스키마에 등록되지 않은 서비스 사용), `invalid_json`(JSON 형식 오류), `temporal_error`(시간 조건 오류)와 같은 실패 유형을 반환한다. 탐색 과정에서는 이 정보를 이용해 관련 블록에 세부 규칙을 추가하거나 기존 블록을 짧고 엄격한 버전으로 바꾼다.

2. 후보 프롬프트의 표현 방식

후보 프롬프트는 고정 길이 이진 문자열이 아니라 여러 설정을 담은 JSON으로 표현한다. 탐색 변수는 다음과 같이 정의한다.

$$C = (A, R, F, S, M_t), \qquad C \in \mathscr{C}, \tag{1}$$

여기서 A는 선택 블록의 활성 여부, R은 블록에 추가하는 세부 규칙, F는 예제 수, S는 생성 전략, M_t는 최대 출력 토큰 수이다. 프롬프트 조립기는 이 설정을 정해진 블록 순서에 따라 결합하여 실제 입력 프롬프트 $p(C)$를 만든다. 부모 식별자와 변이 유형은 후보의 생성 이력을 추적하는 정보이므로 탐색 변수와 분리하여 저장한다.

`temperature`는 다음 토큰을 선택할 때 출력의 무작위성을 조절하는 생성 설정이다. 값이 낮을수록 확률이 높은 토큰을 중심으로 안정적으로 생성한다. 초기 후보 파일에는 0.00, 0.05와 0.10을 저장할 수 있도록 하였으나, 본 실험의 로컬 추론은 탐욕적 생성으로 고정하여 이 값을 사용하지 않았다. 따라서 `temperature`는 실제 탐색 공간에서 제

표 3. 대표 관련 연구군과 JOILang 프롬프트의 로컬 적응 요구사항 비교

	탐색 기반 프롬프트 최적화	피드백 기반 개선	제약 기반 생성 및 규칙 검사	프롬프트 압축 / LLM 서빙	본 연구
대표 문헌	[9–14]	[15, 16]	[17, 18]	[19, 20]	–
블록 단위로 탐색할 수 있는 프롬프트 구성	일부	제한적	–	–	✓
로컬 모델 적응	제한적	제한적	–	제한적	✓
DET 평가 결과를 탐색 기준으로 활용	제한적	–	일부	–	✓
토큰 / 지연 시간 측정	제한적	–	제한적	✓	✓
DET와 실행 비용을 함께 고려한 후보 선택	–	–	–	–	✓
모델별 최종 프롬프트 선택	–	–	–	제한적	✓
서비스 스키마를 따라야 하는 DSL 생성	–	–	일부	–	✓

✓: 해당 요구사항을 명시적으로 다룸; 일부: 관련 기능을 부분적으로 다룸; 제한적: 간접적으로 관련되지만 주된 설계 목표는 아님; –: 해당 요구사항을 다루지 않음.

표 4. JOILang 프롬프트의 블록 구성과 탐색 범위

프롬프트 블록	역할	후보 간 변화	대표 설정
역할·과제 설명	자연어 명령을 JOILang JSON으로 변환하도록 모델의 임무를 규정	고정	과제 역할과 출력 대상
서비스 스키마	전체 스키마에서 찾은 관련 서비스의 정식 명칭, 인자 자료형과 허용값을 제공	입력별 구성(비탐색)	명령마다 관련 항목 선택
JOILang 문법	허용 구문과 금지 구문을 설명	핵심 고정·규칙 탐색	기본 문법과 세부 규칙
시간 조건 규칙	`if`, `wait until`, `clock_delay`, `cron`, `period`의 사용 원칙을 제시	탐색	상태/전이 구분, 단위 규칙
출력 형식	필수 키를 갖춘 하나의 JSON 객체만 출력하도록 제약	핵심 고정·규칙 탐색	설명문 출력 금지
스키마 준수 규칙	등록되지 않은 서비스 사용과 잘못된 인자 생성을 억제	탐색	정식 명칭 사용
명령–코드 예제	프롬프트에 포함할 예제 수를 조정	탐색	예제 1–3개
수정·검증 지침	실패 유형과 관련된 부분만 바꾸는 규칙을 제공	탐색	기능·형식·시간 오류 보정

입력별 구성은 사용자 명령에 따라 달라지지만 같은 평가 항목의 모든 후보에 동일하게 제공되므로 GA가 변경하지 않는다.

외하였고, 최대 출력 토큰 수만 512, 768과 1,024개 가운데 선택하였다.

초기 후보 블록과 규칙 풀을 기준으로 가능한 조합 수는 다음과 같이 나타낼 수 있다.

$$|\mathscr{C}| \approx |\mathscr{A}| \times |F| \times |M_t| \times |S| \times \prod_i |R_i|. \tag{2}$$

여기서 $\mathscr{A}$는 필수 블록을 유지하면서 선택 블록의 사용 여부를 정한 조합의 집합이고, F는 예제 수의 선택지이다. M_t와 S는 각각 최대 출력 토큰 수와 생성 전략의 선택지이며, R_i는 블록 i에 추가할 수 있는 세부 규칙의 조합이다. 식 (2)는 각 선택지가 결합되면서 탐색 범위가 커지는 구조를 나타낸다.

후보 점수는 로컬 LLM이 코드를 생성하고 규칙 기반 평가를 마친 뒤에 확인할 수 있다. 가능한 모든 조합을 시험하면 많은 추론 비용이 들기 때문에, 본 연구는 여러 후보를 함께 유지하면서 프롬프트 블록을 재조합하고 실패한 항목과 관련된 블록을 바꾸는 GA를 적용한다. 다른 탐색 방법과의 비교는 동일한 평가 예산을 둔 후속 실험에서 다룰 수 있다.

IV. GA 기반 PromptOps

그림 2는 본 논문에서 제안하는 GA 기반 JOILang PromptOps 절차의 전체 구조를 나타낸다. 그림에서 선택적 의미 평가, 기존 성능 유지 절차와 과거 실패 명령을 다시 검사하는 리플레이는 현재 정량 실험에 포함하지 않은 후속 확장 단계이다.

그림 2의 왼쪽은 GA 탐색의 입력을 나타낸다. 클라우드 작성 프롬프트와 블록 라이브러리로 블록·생성 설정 후보를 만들고, 각 평가 항목에서는 모든 후보에 같은 장치·기능 정보와 관련 서비스 스키마 항목을 제공한다. 이 문맥 정보는 탐색 변수가 아니다. 과거 실패 명령을 다시 검사하는 리플레이와 사용자 피드백은 배포 후 확장을 위한 입력이며 현재 정량 실험에는 사용하지 않았다.

그림의 가운데는 GA 탐색 루프와 후보 모델 풀을 나타낸다. GA는 성능이 좋은 후보를 고르고(선택), 부모 후보의 블록을 조합하며(교차), 일부 블록·설정을 바꾸고(변이), 중복 제거 뒤 부족한 후보를 채우는 과정(보충)을 반복한다. GPT-4.1-mini는 CloudRef 기준점이고, 실제 최적화 대상은 Qwen2.5-Coder-7B, Llama-3.1-8B, Qwen2.5-Coder-14B의 세 로컬 모델이다.

그림의 오른쪽은 생성 결과를 평가하고 그 결과를 탐색에 되돌리는 과정을 나타낸다. “규칙 기반 검증”은 생성 코드가 서비스 스키마, 출력 형식과 시간 조건을 따르는지 검사하고 항목별 점수와 실패 유형을 산출한다. “배포 지향 적합도”는 규칙 평가 결과와 토큰 수·지연 시간을 합친 후보 점수이다. 적합도가 높은 후보는 다음 세대에 남거나 부모로 선택되고, 실패 유형은 관련 블록을 보강하는 후보를 만들 때 참고한다. 탐색이 끝나면 전체 세대에서 적합도가

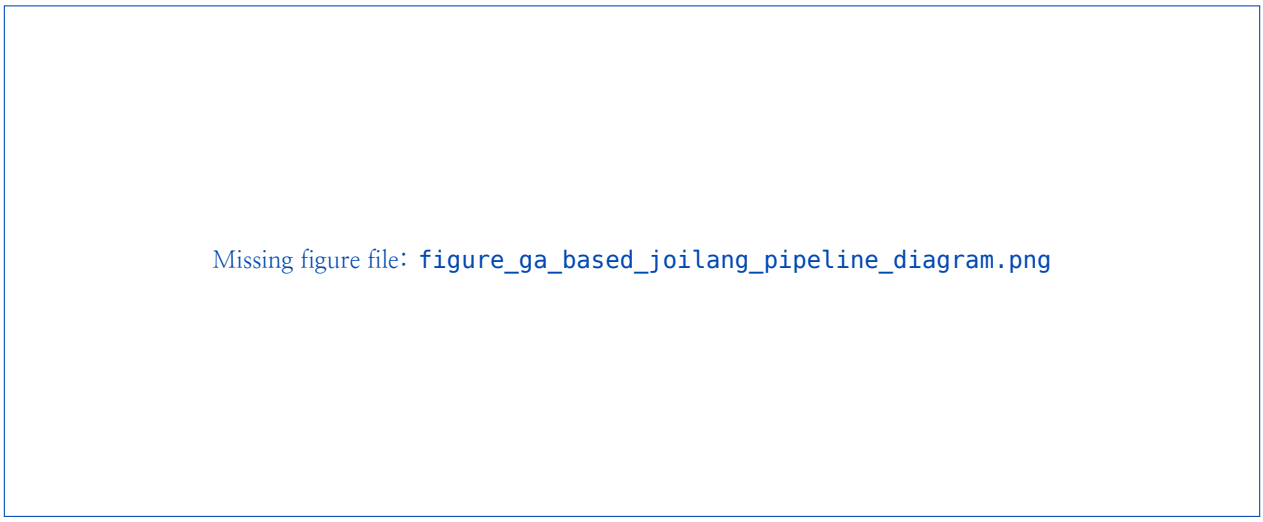


그림 2. GPS-PromptOps의 전체 구조와 운영 확장 지점

가장 높은 후보를 모델별 최종 프롬프트로 선택한다.

그림의 "선택적 의미 평가"는 필요할 때 생성 결과의 의미를 추가로 확인하는 단계이다. "비회귀 수용 게이트"는 새 버전이 현재 배포한 프롬프트의 성능을 떨어뜨리지 않는지 확인하는 기존 성능 유지 절차이다. 프롬프트 버전 관리·배포, 운영 로그와 리플레이도 후속 확장에 해당한다. 이번 실험은 이 단계들을 거치지 않고 DET 평가 결과를 적합도에 직접 반영하였다.

1. 탐색 루프

GA 기반 PromptOps는 초기 프롬프트, 프롬프트 블록 모음, 서비스 스키마, 로컬 대상 모델, 벤치마크 집합과 DET 평가기를 입력으로 받는다. 각 세대에서는 블록과 설정을 실제 프롬프트로 조립하고 로컬 LLM으로 JOILang 코드를 생성한다. 점수가 높은 후보는 그대로 남기고, 일부 후보를 비교해 점수가 높은 후보를 부모로 뽑는다. 이어 부모의 블록을 조합하는 교차와 일부 블록·설정을 바꾸는 변이로 다음 세대 후보를 만든다. 탐색이 끝나면 전체 과정에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 선택한다.

2. 유전 연산

교차는 프롬프트 문자열을 임의 위치에서 자르지 않고 부모 후보의 프롬프트 블록을 조합한다. 자식 후보는 두 부모의 선택 블록, 세부 규칙, 예제 수, 생성 전략과 최대 출력 토큰 수 중 일부를 이어받는다. 예를 들어 한 부모의 스키마 준수 규칙과 다른 부모의 오류 수정 지침을 결합할 수 있다. 의미가 있는 블록 단위로 조합하므로 새 후보도 프롬프트 구조를 유지한다.

변이는 일부 블록이나 설정을 작게 바꾸는 연산이다. 선택 블록을 켜거나 끄고 실패 유형에 맞는 세부 규칙을 추가하며, 예제 수와 생성 전략 및 최대 출력 토큰 수를 바꿀 수 있다. 예를 들어 `unknown_service`에는 서비스 스키마에 등록된 기능만 사용하도록 하는 규칙을, `invalid_json`에는 JSON 전용 출력 규칙을, `temporal_error`에는 시간 전이 규칙을 보강한다. 또한 `canonical_names_first`(정식 명칭 우선)와 같은 생성 전략을 선택하거나 긴 주의 블록을 줄여 토큰 수를 낮출 수 있다.

3. 프롬프트 진화 예시

넓은 탐색 공간에서 교차와 변이만 반복하면 유효한 후보를 찾기 위한 평가 비용이 커질 수 있다. GPS-PromptOps는 일반적인 교차와 변이를 수행하면서, 검증기의 실패 유형을 관련 블록과 세부 규칙을 보강하는 데 활용한다. Qwen2.5-Coder-7B-Instruct가 "5분 후 에어컨을 꺼줘"라는 명령에 대해 지원되지 않는 시간 함수나 잘못된 시간 조건 표현을 포함한 JOILang 후보를 생성하는 경우, 규칙 기반 평가기는 해당 후보를 `temporal_error`로 판정한다.

GA 루프는 이러한 실패 판정 결과를 시간 블록이나 세부 규칙을 보완하는 데 사용한다. 예를 들어 "경과 시간은 JOILang에서 허용한 `clock_delay`와 밀리초 단위로 표현한다"는 규칙을 추가하거나 기존 시간 규칙을 더 명확하게 고칠 수 있다. 이때 시간 표현을 담당하는 블록만 수정한다.

다음 세대에서는 시간 규칙이 보강된 부모 후보가 다른 부모 후보와 교차될 수 있다. 예를 들어 한 부모 후보는 시간 표현의 정확성을 높이지만 프롬프트 길이가 길고, 다른 부모 후보는 토큰 부담이 낮은 간결한 프롬프트 변형을 가질 수 있다. 블록 단위 교차는 두 특성을 문장 단위가 아니라 블록 단위로 결합한다. 그 결과 자식 후보는 올바른 시간 표현 규칙과 더 짧은 프롬프트 구성을 함께 상속할 수 있다.

새 자식 후보도 전체 평가 항목에서 다시 평가한다. 규칙 평가 결과가 좋고 토큰 수와 지연 시간이 낮을수록 높은 적합도를 받는다. 전체 탐색 과정에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 선택한다.

4. 적합도와 DET 피드백

벤치마크의 평가 항목 i에는 비교 기준이 되는 JOILang 코드 r_i가 대응된다. 프롬프트 P와 모델 M이 생성한 코드를 $y_i(P,M)$라 하자. DET 평가는 두 코드를 여덟 항목으로 비교한다. 비교값 s_{ik}는 코드 문자열, 실행 일정, 필요한 서비스의 포함 비율, 불필요한 서비스 호출을 제외한 비율, 대상 태그, 수치값, 문자열·열거값과 값이 대입된 변수 선언문 수의 일치도를 각각 $[0,1]$ 범위로 나타낸다. 평가 항목

i의 DET 점수는 다음과 같다.

$$e_i(P,M) = \sum_{k=1}^{8} w_k\, s_{ik}\big(y_i(P,M), r_i\big), \qquad \sum_{k=1}^{8} w_k = 1. \tag{3}$$

구현에 사용한 가중치 벡터는 비교 항목의 나열 순서대로 $(0.30, 0.15, 0.15, 0.10, 0.10, 0.10, 0.05, 0.05)$이다. 실행 일정, 서비스 호출, 대상과 수치처럼 실행 결과에 직접 영향을 주는 차이를 문자열 표현이나 단순 변수 선언 수의 차이보다 크게 반영하도록 탐색 전에 정한 설계값이다. 예비 실험으로 학습한 값은 아니며 모든 모델에 동일하게 적용하였다. 마지막 항목은 값이 대입된 변수 선언문의 개수만 비교하는 보조 항목으로, 변수 사이의 실제 사용 관계를 분석하지는 않는다. 이 가중치 배분이 결과 해석에 주는 영향은 V장 4절의 민감도 분석에서 정량적으로 확인한다.

JSON 파싱 실패나 기준·생성 코드 누락처럼 점수를 계산할 수 없는 오류가 없고 $e_i(P,M) \geq \tau$이면 해당 항목을 통과로 판정한다. 본 실험의 임계값은 $\tau = 0.70$이다.

$$D_i(P,M) = \begin{cases} 1, & e_i(P,M) \geq \tau \text{ 이고 치명 오류가 없는 경우,} \\ 0, & \text{그 밖의 경우.} \end{cases} \tag{4}$$

전체 평가 항목 수를 N이라 할 때, 프롬프트 P와 모델 M의 DETPass는 다음과 같다.

$$\mathrm{DETPass}(P,M) = \frac{1}{N}\sum_{i=1}^{N} D_i(P,M). \tag{5}$$

DETPass는 규칙 평가를 통과한 항목의 비율이다. 자동 통과 출력은 자연어 명령, 장치·기능 정보 및 기준 코드와 대조하여 사후 검토했으며, 이 검토로 기준 코드나 자동 계산 결과를 바꾸지는 않았다.

프롬프트 P와 모델 M의 S_{DET}는 모든 평가 항목의 DET 점수를 평균한 값이다.

$$S_{\mathrm{DET}}(P,M) = \frac{1}{N}\sum_{i=1}^{N} e_i(P,M). \tag{6}$$

DETPass는 통과한 항목의 비율을 보여 주지만 통과 항목 수가 같은 후보는 구분하지 못한다. S_{DET}는 각 항목의 점수를 남기므로 같은 DETPass를 얻은 후보와 임계값에 가까운 실패 후보를 더 세밀하게 비교할 수 있다. 실패 유형은 관련 블록을 바꾸는 후보를 만들 때 참고한다.

동일한 자동화 시나리오는 실행 결과가 같은 여러 코드로 표현될 수 있다. 공백이나 JSON 직렬화처럼 의미와 무관한 차이는 비교 전에 제거하며, 서비스·대상·시간 순서와 인자 의미가 같은 출력은 기준 코드와 형태가 달라도 DET 점수가 임계값을 넘으면 통과할 수 있다.

실제 평가 항목 "외부 온도를 스피커로 출력해 줘"에서 기준 코드는 WeatherProvider의 온도 값 `temp`를 "외부 온도는 [temp]도입니다"라는 문장으로 전달하고, 간결형은 `speaker_speak(temp)`로 값만 전달한다. 이 항목에서는 문구와 단위 표기를 필수 조건으로 두지 않았으므로 두 출력 모두 통과하였다. 0−1 범위의 DET 점수 e_i는 각각 1.0000과 0.9264였으며, 모두 임계값 이상이므로 $D_i = 1$이다. 따라서 두 출력은 DETPass에 같게 반영되지만, 전달 정보의 구체성과 기준 코드와의 일치도 차이는 S_{DET}에 남는다.

"안방 온도를 23도로 해 줘"에서는 사용자 지정 범위 태그 `#Bedroom`만 쓴 표현과 서비스 유형 태그 `#AirConditioner`를 함께 쓴 표현이 같은 장치를 가리킬 수 있다. 두 표현 모두 통과할 수 있지만, 자연어에 직접 나타난 태그를 사용한 기준 코드와 더 가까운 표현이 높은 DET 점수를 받는다. 사용자 통지 문구의 어투나 같은 동작을 나타내는 호출 형태에서도 이러한 차이가 생길 수 있다.

반면 제어 동작의 방향(켜기와 끄기 등), 사건 대기와 현재 상태 조건, 핵심 수치·열거값처럼 실행 결과를 바꾸는 차이는 허용하지 않는다. S_{DET}는 사용자 선호도가 아니라 기준 코드와의 평균 일치도를 나타낸다.

후보 C의 토큰 수와 지연 시간을 같은 모델의 Direct-Transfer 값으로 나눈 값을 각각 $\mathrm{Tok}(C,M)$와 $\mathrm{Lat}(C,M)$라 한다. 탐색 적합도는 다음과 같이 계산한다.

$$\begin{aligned} f(C,M) = {} & 0.70\,\mathrm{DETPass}\big(p(C),M\big) + 0.20\,S_{\mathrm{DET}}\big(p(C),M\big) \\ & - 0.05\,\mathrm{Tok}(C,M) - 0.05\,\mathrm{Lat}(C,M). \end{aligned} \tag{7}$$

DETPass를 가장 크게 반영하고, 통과 항목 수가 같은 후보는 S_{DET}로 구분하되 토큰 수와 지연 시간이 지나치게 커지는 것을 줄이도록 계수를 정하였다. 이 계수도 탐색 전에 직관적으로 정한 설계값이며 모든 모델에 동일하게 적용하였다. 식 (7)의 0.70은 식 (4)의 통과 임계값과 역할이 다른 별도 설계값이다. DirectTransfer는 비용 항의 정규화와 최종 결과 비교에만 사용한다. 계수 배분이 후보 선발에 주는 영향도 V장 4절에서 분석한다.

규칙 기반 평가기는 점수와 함께 실패 유형을 제공한다. JSON 형식 오류에는 출력 형식 규칙을, 등록되지 않은 서비스 사용에는 정식 명칭 사용 규칙을, 인자 오류에는 자료형과 허용값 규칙을, 시간 조건 오류에는 시간 표현 규칙을 보강한다. 이 정보는 관련 블록을 바꾸는 후보를 만들 때 사용된다.

5. 세대별 후보 선택과 최종 프롬프트 결정

각 세대에서는 적합도가 높은 후보를 엘리트로 남긴다. 부모는 일부 후보를 비교하여 그중 점수가 높은 후보를 고르는 토너먼트 방식으로 선택하고, 블록 단위 교차와 변이를 수행하였다. 이 과정은 이전 세대의 좋은 구성을 보존하면서 새로운 블록 조합을 탐색하기 위한 것이다. 탐색이 끝나면 전체 세대에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 결정하였다.

V. 실험 결과

1. 실험 설정과 모델 범위

본 실험은 제안하는 GA 기반 PromptOps 파이프라인이 모델 가중치를 업데이트하지 않고도 클라우드 작성 JOILang 프롬프트를 로컬 LLM 배포 환경에 맞게 조정할 수 있는지를 평가한다. 평가는 Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, Qwen2.5-Coder-14B-Instruct의 세 로컬 모델에서 수행하였다. 각 로컬 모델에는 동일한 초기 프롬프트, 블록 라이브러리, 탐색 절차와 평가 조건을 적용하고, 모델별로 독립적인 GA 탐색을 수행하여 최종 프롬프트를 선택하였다. 비교 대상인 CloudRef는 원 프롬프트를 GPT-4.1-mini에서 실행한 클라우드 기준 구성이다. 반면 DirectTransfer는 CloudRef 프롬프트를 별도의 모델별 적응 과정 없이 로컬 대상 모델에 직접 적용한 기준 구성이다. 이 구성은 클라우드 작성 프롬프트를 로컬 모델에 그대로 사용하는 경우의 비교 대상이다. 본 실험은 동일한 초기 프롬프트와

표 5. 세 로컬 모델에서 DirectTransfer와 GPS-PromptOps의 최종 비교

모델	구성	DETPass (%)	S_{DET} (DET 점수 평균)	평균 토큰 수 (입력+출력, 항목당)	지연 시간 (평균, s)
GPT-4.1-mini (Cloud)	CloudRef	94.5	0.89	3,500	2.85
Qwen2.5-Coder-7B	DirectTransfer	87.8	0.73	3,500	2.28
	GPS-PromptOps	**92.6**	**0.87**	**1,120**	**1.25**
Llama-3.1-8B	DirectTransfer	81.7	0.68	3,500	2.46
	GPS-PromptOps	**88.7**	**0.82**	**1,250**	**1.40**
Qwen2.5-Coder-14B	DirectTransfer	90.1	0.77	3,500	2.88
	GPS-PromptOps	**94.0**	**0.89**	**1,290**	**1.63**
대상 모델 평균	DirectTransfer	86.5	0.73	3,500	2.54
	GPS-PromptOps	**91.8**	**0.86**	**1,220**	**1.43**

평균값과 모델별 개선 폭은 반올림 전 값으로 계산하였으므로, 표에 표시한 평균값의 단순 차이와 0.1 %p 차이가 날 수 있다.

평가 조건에서 GPS-PromptOps가 DirectTransfer보다 높은 점수를 얻는 모델별 프롬프트를 찾는지 확인한다.

정량 평가에는 JOICommands-280의 자연어 명령 280개를 사용하였다. 범주 1–6은 각각 30개, 범주 7과 8은 각각 50개이며, 단일 장치 제어부터 상태 조건, 지연, 일정과 반복을 포함한 자동화 명령까지 포함한다. 각 자연어 명령에는 해당 명령을 처리할 때 제공하는 장치·기능 정보와 비교 기준 JOILang 코드가 대응된다. 본 논문에서 평가 항목은 이 세 요소를 묶은 하나의 명령 사례를 뜻한다. 각 후보는 앞 절에서 정의한 DET 평가와 비용 측정 절차로 평가하고, 전체 탐색에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 선택하였다. 평가 항목당 평균 토큰 수는 입력과 출력 토큰을 합산한 뒤 280개 항목에서 평균하였다. 모델마다 토큰화 방식이 다르므로 토큰 수는 같은 모델의 DirectTransfer 대비 변화로 비교하였다. 후보 탐색과 최종 비교에는 같은 벤치마크 집합을 사용했으므로, 보고 수치는 탐색에 사용하지 않은 독립 시험 집합의 성능으로 해석하지 않는다.

2. 비교 구성과 전체 성능

표 5에 세 로컬 모델에서 측정한 최종 결과를 제시한다. 표의 각 행은 모델과 구성의 조합이고, 열은 DETPass, S_{DET}, 평가 항목당 평균 토큰 수와 지연 시간이다.

표 5의 GPS-PromptOps 결과는 각 대상 모델을 위해 선택한 최종 프롬프트를 해당 모델에서 평가한 값이다. 모델 간 교차 적용 결과는 표 6에서 별도로 비교한다.

세 로컬 모델 모두에서 GPS-PromptOps의 DETPass는 DirectTransfer보다 높았으며, 모델별 차이는 3.9–7.0 %p, 평균 차이는 5.2 %p였다. 세 대상 모델의 평균으로 계산하면 DETPass는 86.5%에서 91.8%로, S_{DET}는 0.73에서 0.86으로 증가하였다. 평가 항목당 평균 토큰 수는 3,500에서 1,220으로 65.1%, 평균 지연 시간은 2.54초에서 1.43초로 43.7% 감소하였다. 블록 구성과 최대 출력 토큰 수를 함께 탐색했으므로, 지연 시간 감소를 토큰 수 감소만의 효과로 해석하지 않는다.

모델별 DETPass 개선 폭은 DirectTransfer에서 가장 낮은 값을 보인 Llama-3.1-8B에서 7.0 %p로 가장 컸다. DirectTransfer 성능이 가장 높았던 Qwen2.5-Coder-14B는 향상 폭이 3.9 %p로 상대적으로 작았으나, 최종 DETPass 94.0%로 CloudRef(94.5%)와의 격차를 0.5 %p까지 좁혔다.

표 6. 벤치마크 집합에서 모델별 선택 프롬프트의 교차 적용 결과 (DETPass, %)

적용 프롬프트	Qwen-7B	Llama-8B	Qwen-14B
DirectTransfer	87.8	81.7	90.1
P_{Q7}	**92.6**	84.9	91.8
P_{L8}	89.3	**88.7**	91.2
P_{Q14}	90.4	81.2	**94.0**
Δ_{self} (%p)	2.2	3.8	2.2

CloudRef의 지연 시간은 클라우드 API 환경에서 측정되었으므로 로컬 모델과 비교하지 않았으며 로컬 모델의 비용 개선은 각 모델에서 DirectTransfer와 GPS-PromptOps의 차이를 구하였다.

3. 모델 간 프롬프트 교차 적용

DirectTransfer 대비 성능이 높아졌다는 결과만으로는 탐색된 프롬프트가 모델별로 특화되었다고 판단하기 어렵다. 하나의 공통 프롬프트가 세 모델 모두에서 가장 좋은 결과를 낼 수도 있기 때문이다. 이를 확인하기 위해 Qwen2.5-Coder-7B, Llama-3.1-8B, Qwen2.5-Coder-14B용으로 각각 선택한 최종 프롬프트 P_{Q7}, P_{L8}, P_{Q14}를 세 대상 모델에 교차 적용하였다.

표 6에서 세 대상 모델은 모두 해당 모델용 프롬프트를 적용했을 때 가장 높은 DETPass를 보였다. Δ_{self}는 해당 모델용 프롬프트와 가장 높은 타 모델용 프롬프트의 차이이다. 이 값은 Qwen-7B에서 2.2 %p, Llama-8B에서 3.8 %p, Qwen-14B에서 2.2 %p였으며 평균은 2.7 %p이다. 어느 하나의 프롬프트도 세 모델에서 공통으로 최고값을 보이지는 않았다.

교차 적용 결과는 모델 조합에 따라 달랐다. P_{Q14}를 Qwen-7B에 적용했을 때 DETPass는 90.4%, P_{Q7}를 Qwen-14B에 적용했을 때는 91.8%였다. 반면 P_{Q14}를 Llama-8B에 적용한 결과는 81.2%로 DirectTransfer의 81.7%보다 0.5 %p 낮았다. 다른 모델에서 선택한 프롬프트가 항상 성능을 높이지는 않았다. 이 분석은 동일 벤치마크 집합에서 수행했으므로, 독립 시험 집합의 일반화 성능과 통계적 유의성은 별도 검증이 필요하다.

4. 평가 가중치와 적합도 계수의 민감도 분석

식 (3)의 가중치 w_k와 식 (7)의 계수는 탐색 전에 정한 설계값이므로, 이 값이 달라지면 보고한 비교 결과가 어떻게 변하는지 확인할 필요가 있다. 이를 위해 저장된 생성 결과에서 여덟 비교값 s_{ik}를 추출한 뒤 가중치 변형별로 e_i를 다시 계산하는 재채점 방식의 민감도 분석을 수행하였다. 재채점은 코드 생성을 반복하지 않으므로, 이 분석은 가중치 변화가 판정과 순위에 주는 영향을 측정하며 탐색 자체를 다시 실행한 결과는 아니다. 데이터는 GPT-4.1-mini 기반 프롬프트 버전 5종과 Qwen2.5-Coder-14B의 280개 항목 평가 기록 6종을 사용하였다.

가중치 변형은 세 층위로 구성하였다. (1) 배분 전체를 바꾸는 프로파일 4종(균등, 실행 강조, 구조 강조, 코드 유사도 제외), (2) 한 항목씩 제거하고 재정규화하는 LOO(leave-one-out) 8종, (3) 각 항목을 ±20% 변형하고 재정규화하는 16종이다. 판정 기준은 재채점 실행 전에 고정하였다: 기준 배분 대비 DETPass 변화의 최대 절댓값이 2 %p 이하이고 프롬프트 버전 간 순위 상관(Spearman ρ)이 0.90 이상이며 항목별 통과 판정 반전율이 5% 이하이면 강건(strong), 각각 5 %p·0.80·10% 이내이면 보통(moderate), 그 밖에는 민감(sensitive)으로 분류한다.

표 7에 결과를 요약한다. ±20% 변형 16종은 전부 강건으로 분류되었다(최대 변화 1.07 %p, 순위 상관 ≥ 0.96). 즉 보고한 비교 결과는 가중치의 국소적 변화에 사실상 불변이다. 반면 LOO에서는 큰 가중치를 받은 항목일수록 제거의 영향이 컸다. 서비스 포함 비율 제거는 최대 6.07 %p, 코드 문자열·실행 일정·서비스 정밀도 제거도 민감으로 분류된 반면, 가중치 $0.05-0.10$의 보조 항목(대상 태그, 열거값, 변수 선언 수) 제거는 강건 범위에 머물렀다. 이 대비는 식 (3)의 배분이 실행에 직접 영향을 주는 항목에 큰 가중치를 준 설계와 실제 판별 기여가 서로 일치함을 보여준다. 배분 전체를 바꾸는 프로파일 교체는 예상대로 민감 범위였다(균등 4.29 %p, 코드 유사도 제외 5.00 %p).

표 8는 기준 배분에서 τ를 0.65, 0.70, 0.75로 바꾼 결과이다. DETPass의 절대값은 τ에 따라 단조로 변하지만, 여섯 평가 대상 사이의 우열 순서는 대체로 유지되었다. 따라서 $\tau = 0.70$은 절대 수치에는 영향을 주는 선택이지만 비교 결론을 좌우하는 선택은 아니다.

표 9는 식 (7)의 계수 배분을 바꾸었을 때 후보 선발이 어떻게 달라지는지를 보여 준다. 보존된 Qwen2.5-Coder-14B GA 탐색 기록의 세대별 후보 평가값에 다섯 가지 계수 프로파일을 적용해 재채점하였다. DETPass와 S_{DET} 사이의 비중을 바꾸는 변형(통과율 강조, 점수 강조)은 세대 내 최우수 후보를 바꾸지 않았다(순위 상관 ≥ 0.94). 반면 토큰·지연 계수를 0.05에서 0.20으로 키우면 최우수 후보가 70%의 세대에서 바뀌었다. 즉 품질 항 사이의 세부 배분은 선발에 거의 영향을 주지 않는 반면, 비용 항의 크기는 선발을 좌우한다. 이는 0.05의 작은 비용 계수가 "품질이 비슷한 후보 중 저비용 후보를 선호"하는 보조 역할을 하도록 정한 설계 의도와 일치한다.

요약하면, 보고한 결과는 설계값 근방의 가중치·계수 변화에 강건하며, 큰 가중치를 받은 비교 항목일수록 제거 시 영향이 커서 배분의 우선순위와 실제 판별 기여가 일치한다. 이 분석은 저장된 생성 결과의 재채점에 기반하므로, 가중치 변형별로 GA 탐색을 처음부터 다시 실행하는 검증은 후속 연구 과제이다.

표 7. DET 가중치 변형에 대한 재채점 민감도 요약. 기준 배분 대비 DETPass 변화(최대·평균), 프롬프트 버전 순위 상관, 항목별 판정 반전율과 사전 고정 기준에 따른 판정이다.

가중치 변형	max|ΔDETPass|(pp)	평균 Δ(pp)	Spearman ρ	max flip	판정
균등	4.29	2.62	0.700	4.3%	sensitive
실행 강조	2.14	0.65	0.900	2.9%	moderate
구조 강조	4.64	−2.68	0.600	6.4%	sensitive
코드 유사도 제외	5.00	1.85	0.400	5.0%	sensitive
LOO 8종 최대(=service_coverage)	6.07	−0.06	0.900	8.9%	sensitive
±20% 16종 최대(=p20_schedule)	1.07	0.24	1.000	2.1%	strong

표 8. 통과 임계값 τ에 따른 DETPass(%) 변화. 열은 프롬프트 버전 5종(GPT-4.1-mini)과 Qwen2.5-Coder-14B이다.

τ	cloud_v07	cloud_v12	cloud_v13	cloud_v14	cloud_v15	qwen14b
0.65	96.1	95.0	97.5	96.8	96.8	88.6
0.70	92.5	91.4	95.0	92.9	92.1	86.1
0.75	87.9	84.3	89.3	89.3	84.3	83.9

표 9. 적합도 계수 변형이 후보 선발에 주는 영향. 보존된 GA 탐색 기록의 세대별 후보를 계수 프로파일별로 재채점하여 세대 내 최우수 후보 변경률과 순위 상관(Kendall τ)을 측정하였다.

계수 프로파일	$(\alpha, \beta, \lambda_t, \lambda_l)$	우승자 변경률	평균 Kendall τ	평균 적합도
기본 (식 7)	(0.7, 0.2, 0.05, 0.05)	0.0%	1.000	0.325
통과율 강조	(0.85, 0.1, 0.025, 0.025)	0.0%	1.000	0.398
점수 강조	(0.45, 0.45, 0.05, 0.05)	0.0%	0.940	0.323
비용 강조	(0.5, 0.1, 0.2, 0.2)	70.0%	0.320	−0.105
품질 전용	(0.78, 0.22, 0, 0)	20.0%	0.760	0.468

VI. 결론

이 논문에서 우리는 클라우드에서 작성한 JOILang 프롬프트를 로컬 LLM별로 GA 알고리즘을 이용하여 최적화하는 GPS-PromptOps를 제안하였다. 후보 프롬프트는 선택 블록, 세부 규칙, 예제 수와 생성 설정의 조합으로 표현한다. DET 점수와 실패 유형은 후보 선택과 변이에 사용하고, 탐색 과정에서 적합도가 가장 높은 후보를 모델별 최종 프롬프트로 선택하였다.

세 로컬 모델(Qwen2.5-Coder-7B, Llama-3.1-8B, Qwen2.5-Coder-14B)에서 GPS-PromptOps의 DETPass는 DirectTransfer보다 평균 5.2 %p 높았고, 평가 항목당 평균 토큰 수와 지연 시간은 각각 65.1%와 43.7% 감소하였다. 교차 적용에서도 각 모델은 해당 모델용 프롬프트에서 가장 높은 DETPass를 기록했으며, 두 타 모델용 프롬프트 중 더 높은 결과보다 평균 2.7 %p 높았다. 저장된 생성 결과를 재채점한 민감도 분석에서는 평가 가중치의 ±20% 변형에 대해 비교 결과가 강건했고, 적합도의 품질 항 배분 변화는 후보 선발을 거의 바꾸지 않았다. 이 결과는 평가한 모델과 벤치마크 범위에서 로컬 모델별 프롬프트 조정의 필요성을 보여 준다.

본 실험은 탐색과 최종 비교에 같은 벤치마크 집합을 사용했고, 실제 장치 실행 대신 정적 DET 평가를 적용하였다. 후속 연구에서는 탐색에 사용하지 않은 시험 집합과 독립 반복 실행을 통해 모델별 프롬프트의 일반화와 탐색 변동을 검증할 계획이다. 또한 Phi-3.5-mini와 Gemma-2-9B처럼 문맥 길이 제약이 큰 모델을 대상으로 프롬프트 압축을 포함한 탐색으로 확장할 예정이다.

References

[1] H. J. Lee, E. J. Jeong, D. Kang, J. Kim, and S. Ha, "Work-in-progress: SoPIoT: Service-oriented platform for internet of things," in *Proceedings of the 13th ACM International Conference on Embedded Software 2017 Companion*, 2017.

[2] M. Jeong, E. Jeong, and S. Ha, "JOILang: Reliable natural-language-to-DSL generation for service-level IoT automation," 2026. Manuscript.

[3] E. Kim, T. Son, and S. Ha, "A novel hierarchical edge-based architecture for service oriented IoT," *Internet of Things*, vol. 24, p. 100939, 2023.

[4] M. Navardi, R. Aalishah, Y. Fu, Y. Lin, H. Li, Y. Chen, and T. Mohsenin, "GenAI at the edge: Comprehensive survey on empowering edge devices," in *Proceedings of the AAAI Symposium Series*, vol. 5, pp. 180 – 187, 2025.

[5] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications," *arXiv preprint arXiv:2410.11845*, 2024.

[6] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proceedings of the 38th International Conference on Machine Learning*, pp. 12697 – 12706, 2021.

[7] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pp. 8086 – 8098, 2022.

[8] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11048 – 11064, 2022.

[9] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "AutoPrompt: Eliciting knowledge from language models with automatically generated prompts," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 4222 – 4235, 2020.

[10] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in *Proceedings of the 11th International Conference on Learning Representations*, 2023.

[11] A. Prasad, P. Hase, X. Zhou, and M. Bansal, "GrIPS: Gradient-free, edit-based instruction search for prompting large language models," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 3845 – 3864, 2023.

[12] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," *arXiv preprint arXiv:2309.03409*, 2023.

[13] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers," *arXiv preprint arXiv:2309.08532*, 2023.

[14] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktaschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[17] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901, 2021.

[18] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 440–450, 2017.

[19] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13358–13376, 2023.

[20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.

Authors

Mingi Jeong received the M.S. degree in computer science and engineering from Seoul National University, Seoul, Korea. He is currently pursuing a Ph.D. in the Department of Computer Science and Engineering, Seoul National University, Seoul, Korea. He is also a Researcher with the Electronics and Telecommunications Research Institute (ETRI), Daejeon, Korea. His research interests include natural language-driven IoT automation, domain-specific languages, retrieval-augmented generation, and LLM-based code generation.

Eunjin Jeong received the Ph.D. degree in computer science and engineering from Seoul National University, Seoul, Korea. He is currently a Postdoctoral Researcher with Seoul National University, Seoul, Korea. His research interests include automatic robot software synthesis and embedded machine learning.

Soonhoi Ha received the B.S. and M.S. degrees in electronics engineering from Seoul National University, Seoul, Korea, and the Ph.D. degree in electrical engineering and computer science from the University of California at Berkeley, Berkeley, CA, USA. He is currently a Professor with the Department of Computer Science and Engineering, Seoul National University, Seoul, Korea. His research interests include the HW/SW co-design of embedded systems, embedded machine learning, and the Internet of Things.